

CS 2011E — Database Management Systems
Winter Semester 2026 | Class CS02 | Team 1

Urban Smart Parking Management System

Implementation and Testing Document

Prepared by

Name	Roll Number
Farzan Faisal	B240179CS
Elias Joby	B240175CS
Reja Fairooz	B240281CS
Amil K	B240136CS
Ronn Boban	B240286CS
Kona Bhuvaneswar Reddy	B240214CS

April 2026

1. Introduction

1.1 Purpose

This document describes the implementation details and testing strategy for the Urban Smart Parking Management System (USPMS), a full-stack web application developed as a course project for CS 2011E — Database Management Systems, Winter Semester 2026. It covers the system architecture, technology stack, database schema, module implementation, and a comprehensive test plan with test cases and results.

1.2 Scope

USPMS enables urban drivers to discover parking lots, reserve and release slots, and track billing — while giving administrators full control over lots, slots, sessions, and user dues. The application is deployed live at:

<https://urban-smart-parking-system.onrender.com>

1.3 Document Organisation

The remainder of this document is structured as follows:

- Section 2 — System Architecture and Technology Stack
- Section 3 — Database Design and Implementation
- Section 4 — Module Implementation

- Section 5 — API Implementation
- Section 6 — Frontend Implementation
- Section 7 — Deployment Configuration
- Section 8 — Test Plan
- Section 9 — Test Cases and Results
- Section 10 — Known Issues and Limitations
-

2. System Architecture

2.1 Architectural Overview

USPMS follows a classic three-tier client-server architecture:

Tier	Technology	Responsibility
Presentation (Frontend)	HTML, CSS, Vanilla JavaScript	Role-based page routing, API integration, UI rendering
Application (Backend)	Node.js + Express.js	REST API, business logic, reservation management, fee computation
Data (Database)	MySQL via mysql2 (Clever Cloud)	Persistent storage, ACID transactions, auto table creation

2.2 Technology Stack

Component	Technology / Version	Purpose
Frontend	HTML5, CSS3, Vanilla JS	User interface for drivers and admins
Backend Runtime	Node.js 18+ / 22	Server-side JavaScript execution
Web Framework	Express.js	RESTful API routing and middleware
Database	MySQL (mysql2 driver)	Relational data storage
DB Hosting	Clever Cloud MySQL	Cloud-managed MySQL instance
App Hosting	Render (Web Service)	Containerised application deployment
Package Manager	npm	Dependency management

2.3 Repository Structure

The project is organised into clearly separated frontend (public/) and backend (server/) directories:

Urban-Smart-Parking-System/

public/	
index.html	(Landing / home page)
login.html	(Authentication page)
driver.html	(Driver dashboard)
admin.html	(Admin dashboard)
css/	(Stylesheets)
js/	(Client-side scripts)

server/	
server.js	(Express app, all API routes)
database.js	(MySQL pool, table auto-creation)
wipe_db.js	(Dev utility – clears all tables)
Docs/	(Project documentation)
package.json	
README.md	

3. Database Design and Implementation

3.1 Schema Overview

The database is MySQL-hosted on Clever Cloud. All tables are auto-created at server startup via the database.js initialisation module. The schema consists of four core tables:

3.1.1 users

Stores all registered accounts for both drivers and the admin.

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique user identifier
username	VARCHAR(100)	NOT NULL, UNIQUE	Login username
password	VARCHAR(255)	NOT NULL	Plain-text password (see limitations)
role	ENUM	'driver','admin'	Access control role
phone	VARCHAR(20)	NULLABLE	Driver contact number
due_fees	DECIMAL(10,2)	DEFAULT 0.00	Cumulative unpaid balance (INR)

3.1.2 vehicles

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique vehicle identifier
user_id	INT	FK -> users(id)	Owning driver
license_plate	VARCHAR(50)	NOT NULL	Vehicle registration number
vehicle_type	ENUM	'car','bike','truck'	Category of vehicle
make_model	VARCHAR(100)	NOT NULL	Manufacturer and model

3.1.3 parking_lots

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique lot identifier
name	VARCHAR(100)	NOT NULL	Display name of the lot
address	VARCHAR(255)	NOT NULL	Physical location
total_slots	INT	NOT NULL	Total capacity
price_per_hour	DECIMAL(10,2)	NOT NULL	Hourly rate in INR

3.1.4 parking_slots

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique slot identifier
lot_id	INT	FK -> parking_lots(id)	Parent lot
slot_number	VARCHAR(20)	NOT NULL	Human-readable slot label (e.g. A-3)
slot_type	ENUM	'car','bike','truck'	Supported vehicle type
status	ENUM	'available','occupied','reserved'	Current occupancy state
user_id	INT	FK -> users(id), NULLABLE	Driver currently using slot
vehicle_id	INT	FK -> vehicles(id), NULLABLE	Vehicle currently in slot

3.1.5 parking_sessions

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique session identifier
user_id	INT	FK -> users(id)	Driver who parked
vehicle_id	INT	FK -> vehicles(id)	Vehicle used
slot_id	INT	FK ->	Slot occupied

		parking_slots(id)	
lot_id	INT	FK -> parking_lots(id)	Lot where session occurred
entry_time	DATETIME	NOT NULL	Session start timestamp
exit_time	DATETIME	NULLABLE	Session end timestamp
duration_hours	DECIMAL(10,4)	NULLABLE	Computed duration
fee_charged	DECIMAL(10,2)	NULLABLE	Fee permanently stamped on close
status	ENUM	'active','complete'	Current session state

3.2 Referential Integrity and Cascade Behaviour

- Deleting a parking lot cascade-deletes all its associated slots.
- Deleting a user removes all their vehicles and clears any active slot assignments.
- MySQL transactions (BEGIN / COMMIT / ROLLBACK) are used during reservation and release to ensure atomicity and prevent double-booking.

4. Module Implementation

4.1 Authentication Module

Authentication is handled via POST /api/login and POST /api/signup. On successful login the server returns the user ID, username, and role. The frontend stores these in sessionStorage and uses client-side guards to redirect unauthenticated access.

Current implementation uses plain-text credential comparison. Password hashing with bcrypt and JWT-based session tokens are identified as future enhancements.

4.2 Vehicle Management Module

Drivers can register multiple vehicles against their account. Each vehicle record stores the license plate, type (car/bike/truck), and make/model. The driver must have a phone number on file before adding a vehicle. Vehicle deletion cascades to clear any slot references.

4.3 Parking Lot and Slot Management Module

Admins create lots by supplying the lot name, address, total slot count, and hourly rate. The system auto-generates numbered slots (A-1, A-2, ...) on lot creation, all initialised with status = 'available'.

Admins can update slot types individually (car/bike/truck) or force override a slot's status to resolve discrepancies. Lot deletion cascades to delete all contained slots.

4.4 Reservation Module

The reservation flow is the most transaction-critical operation in the system:

1. The driver selects an available slot and a registered vehicle.
2. The server opens a MySQL transaction.

3. The slot row is locked (SELECT ... FOR UPDATE) to prevent concurrent reservations.
 4. If still available, the slot status is set to 'occupied', and user_id / vehicle_id are assigned.
 5. A new parking_sessions record is created with entry_time = NOW() and status = 'active'.
 6. The transaction is committed; the driver's dashboard updates to show the active session.
- If any step fails, the transaction is rolled back and an error is returned to the client.

4.5 Release and Billing Module

When a driver clicks Exit & Complete Session:

7. The server calculates duration: duration_hours = (exit_time - entry_time) in hours.
8. Fee is computed: fee_charged = duration_hours x price_per_hour of the lot.
9. The session record is updated with exit_time, duration_hours, fee_charged, and status = 'completed'.
10. The slot is reset: status = 'available', user_id = NULL, vehicle_id = NULL.
11. The driver's due_fees balance is incremented by fee_charged.

The fee formula is: Fee (INR) = Duration (hours) x Price Per Hour

4.6 Admin Override Module

Administrators have two override capabilities: (a) manual fee adjustment on any driver account via PUT /api/admin/users/:userId/dues, and (b) forced slot status override via PUT /api/admin/slots/:slotId/status. These are intended for resolving operational discrepancies.

5. API Implementation

5.1 Authentication Endpoints

Method	Endpoint	Description
POST	/api/login	Validate credentials; return user object with role
POST	/api/signup	Create a new driver account

5.2 Driver Endpoints

Method	Endpoint	Description
POST	/api/user/phone	Save driver phone number
GET	/api/vehicles/:userId	List driver vehicles
POST	/api/vehicles	Register a new vehicle
DELETE	/api/vehicles/:id	Remove a vehicle
GET	/api/lots	List all parking lots with availability
GET	/api/lots/:id/slots	List slots for a specific lot
POST	/api/reserve	Reserve a slot (atomic transaction)
POST	/api/release	Release slot and compute fees

GET	/api/my-sessions/:userId	Get driver's session history
GET	/api/user/dues/:userId	Get driver's current due fees

5.3 Admin Endpoints

Method	Endpoint	Description
GET	/api/admin/users	List all drivers with vehicle and session info
GET	/api/admin/sessions	List all parking sessions
POST	/api/lots	Create a new parking lot with auto-generated slots
DELETE	/api/lots/:id	Delete lot and cascade-delete all its slots
PUT	/api/slots/:slotId	Update slot type configuration
PUT	/api/admin/slots/:slotId/status	Force-override slot status
DELETE	/api/admin/users/:id	Delete a driver account with cleanup
PUT	/api/admin/users/:userId/dues	Manually set or adjust a driver's due fees

6. Frontend Implementation

6.1 Page Structure

The frontend is composed of four static HTML pages served from the public/ directory. Client-side JavaScript handles all API calls, role-based routing, and dynamic UI updates without any page reload framework.

Page	Purpose
index.html	Landing page — entry point, redirects based on session state
login.html	Authentication — handles both login and driver sign-up forms
driver.html	Driver Dashboard — vehicle management, lot browsing, reservations, session history, dues
admin.html	Admin Dashboard — lot/slot management, session monitoring, user management, fee overrides

6.2 Role-Based Access Control

After login, the server response includes a role field ('driver' or 'admin'). The JavaScript layer stores this in sessionStorage and guards every dashboard-level action client-side.

Unauthenticated or wrong-role requests are redirected to login.html.

6.3 Real-Time Availability Display

Slot availability is fetched from the server each time the driver opens a lot view. Green-bordered slots indicate 'available'; slots with other statuses are visually distinguished. Atomic transactions on the server side ensure the state seen by one client does not conflict with another concurrent reservation.

7. Deployment Configuration

7.1 Hosting

The application is deployed on Render as a Node.js web service. The MySQL database is hosted on Clever Cloud. Communication between the app server and the database uses the DATABASE_URL environment variable.

7.2 Environment Variables

Variable	Purpose
DATABASE_URL	Clever Cloud MySQL connection URI (includes host, port, credentials, db name)
PORT	HTTP port — Render sets this automatically; default 10000
NODE_ENV	Set to 'production' on Render; affects error verbosity

7.3 Build and Start Commands

- Build Command: npm install
- Start Command: node server/server.js
- Node version requirement: 18+ (Node 22 also tested and confirmed compatible)

8. Test Plan

8.1 Testing Approach

Testing was conducted as functional black-box testing against the deployed application at <https://urban-smart-parking-system.onrender.com>. Each test case was exercised manually by team members acting in the specified user role. The test design follows equivalence partitioning and boundary value analysis principles.

8.2 Test Environment

- Browser: Google Chrome (latest), Mozilla Firefox (latest)

- Network: Standard broadband connection
- Database: Clever Cloud MySQL (live production instance)
- Test Execution Date: April 2026

8.3 Test Coverage Areas

- Authentication — registration, login, logout, invalid credentials
- Vehicle Management — add, view, delete vehicles
- Parking Lot Management — create, edit, delete lots (admin)
- Parking Slot Management — slot type updates, status overrides
- Reservation — normal flow, concurrent booking prevention
- Session Release and Billing — fee calculation accuracy
- Due Fees — balance tracking and admin override
- Session History — record completeness
- Role Enforcement — cross-role access prevention

9. Test Cases and Results

9.1 Authentication Tests

ID	Test Case	Steps	Expected	Actual	Result
TC-A01	Driver Registration	Navigate to Sign Up; enter unique username and password; submit.	Account created; redirected to login page.	As expected.	PASS
TC-A02	Duplicate Username Registration	Attempt to register with an already-existing username.	Error message: username already exists.	As expected.	PASS
TC-A03	Valid Driver Login	Enter correct driver credentials; click Login.	Redirected to Driver Dashboard.	As expected.	PASS
TC-A04	Valid Admin Login	Enter admin credentials; click Login.	Redirected to Admin Dashboard.	As expected.	PASS
TC-A05	Invalid Login — Wrong Password	Enter valid username with incorrect password.	Error message returned; no redirect.	As expected.	PASS
TC-A06	Logout	Click Logout button on dashboard.	Session cleared; redirected to login page.	As expected.	PASS

9.2 Vehicle Management Tests

ID	Test Case	Steps	Expected	Actual	Result
TC-V01	Add Vehicle — Valid	Log in as driver; save phone number; click Add Vehicle; fill all fields; submit.	Vehicle appears in My Vehicles list.	As expected.	PASS
TC-V02	Add Vehicle — Missing Phone	Attempt to add vehicle without saving a phone number.	System prompts driver to save phone number first.	As expected.	PASS

TC-V03	Add Vehicle — Missing Fields	Submit Add Vehicle form with one or more empty fields.	Submission blocked; form validation error.	As expected.	PASS
TC-V04	View Vehicles	Navigate to My Vehicles after adding two vehicles.	Both vehicles listed with correct details.	As expected.	PASS
TC-V05	Delete Vehicle	Click Delete on a registered vehicle.	Vehicle removed from list.	As expected.	PASS

9.3 Parking Lot Management Tests (Admin)

ID	Test Case	Steps	Expected	Actual	Result
TC-L01	Create Parking Lot	Admin: navigate to Parking Lots; fill in name, address, slot count, price; click Create Lot.	Lot created; slots auto-generated and listed.	As expected.	PASS
TC-L02	Create Lot — Missing Fields	Submit Create Lot form with empty required fields.	Submission blocked; error message shown.	As expected.	PASS
TC-L03	Edit Parking Lot	Click Edit on an existing lot; modify price per hour; save.	Updated price reflected immediately.	As expected.	PASS
TC-L04	Delete Parking Lot	Click Delete on a lot with no active sessions.	Lot and all its slots removed from system.	As expected.	PASS
TC-L05	Slot Type Configuration	Admin: select a slot; update type from 'car' to 'bike'.	Slot type updated; visible to drivers.	As expected.	PASS
TC-L06	Force Slot Status Override	Admin: force-set an occupied slot back to 'available'.	Slot status updated immediately.	As expected.	PASS

9.4 Reservation and Session Tests

ID	Test Case	Steps	Expected	Actual	Result
TC-R01	Reserve a Slot — Valid	Driver: find parking; click available slot; select vehicle; confirm.	Slot becomes occupied; active session shown on dashboard.	As expected.	PASS
TC-R02	Reserve Already-Occupied Slot	Driver: attempt to click an occupied slot.	Slot not selectable; no reservation created.	As expected.	PASS
TC-R03	Reserve Without Vehicle	Driver: attempt to confirm reservation with no vehicles registered.	Error: no vehicles available; reservation blocked.	As expected.	PASS
TC-R04	View Active Session	After reserving, check dashboard active session panel.	Lot name, slot number, entry time, and vehicle displayed correctly.	As expected.	PASS
TC-R05	Release Slot	Driver: click Exit & Complete Session on active session.	Session completed; slot returned to available; dues updated.	As expected.	PASS
TC-R06	Fee Calculation Accuracy	Reserve a slot at Rs.2.30/hr; release after approximately 1 hour.	Fee of approx. Rs.2.30 added to due_fees.	As expected.	PASS

TC-R07	Session History Record	After completing a session, view Session History.	Record shows correct entry time, exit time, duration, fee, lot, and slot.	As expected.	PASS
--------	------------------------	---	---	--------------	------

9.5 Billing and Due Fees Tests

ID	Test Case	Steps	Expected	Actual	Result
TC-B01	Due Fees Balance Display	View driver dashboard after completing a session.	Due fees balance shown correctly in INR.	As expected.	PASS
TC-B02	Due Fees Accumulation	Complete two separate sessions; verify total.	Total due equals sum of both session fees.	As expected.	PASS
TC-B03	Admin Fee Override — Set to Zero	Admin: locate driver; click Adjust Fees; set balance to 0.	Driver's due_fees updated to Rs.0.00.	As expected.	PASS
TC-B04	Admin Fee Override — Custom Amount	Admin: set a driver's due_fees to a custom amount.	Balance updated immediately; visible on driver dashboard.	As expected.	PASS
TC-B05	Admin View All Sessions	Admin: navigate to Sessions panel.	All active and completed sessions listed with correct details.	As expected.	PASS

9.6 Role and Access Enforcement Tests

ID	Test Case	Steps	Expected	Actual	Result
TC-X01	Driver Cannot Access Admin Dashboard	Log in as driver; manually navigate to admin.html.	Redirected away or admin features not accessible.	Redirected to login.	PASS
TC-X02	Admin Cannot Create Reservations	Log in as admin; driver reservation controls absent.	No reservation UI available on admin dashboard.	As expected.	PASS
TC-X03	Unauthenticated Access Blocked	Navigate directly to driver.html without logging in.	Redirected to login page.	As expected.	PASS
TC-X04	Admin User Deletion	Admin: locate a driver; click Delete user.	Driver account removed; any active slot usage cleared.	As expected.	PASS

Screenshots of Testing, appended at the end. (10.4)

9.7 Test Summary

Metric	Value
Total Test Cases	27
Passed	27
Failed	0

Pass Rate	100%
-----------	------

10. Known Issues and Limitations

10.1 Security Limitations

- Passwords are stored in plain text. Production hardening requires bcrypt hashing.
- Authentication uses plain credential comparison with no token-based sessions. JWT-based authentication is planned for a future version.
- Role-based access control is primarily enforced client-side. Backend middleware guards should be added before any production deployment.

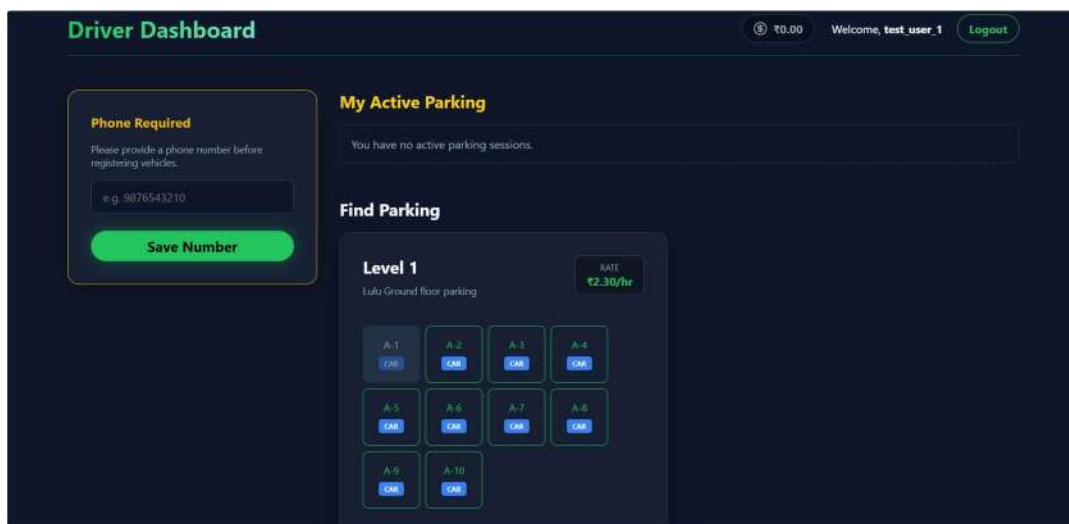
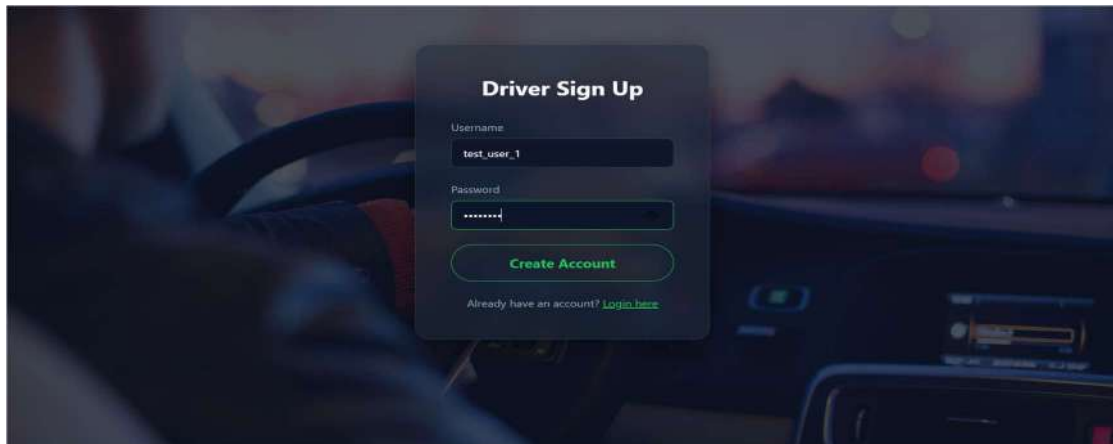
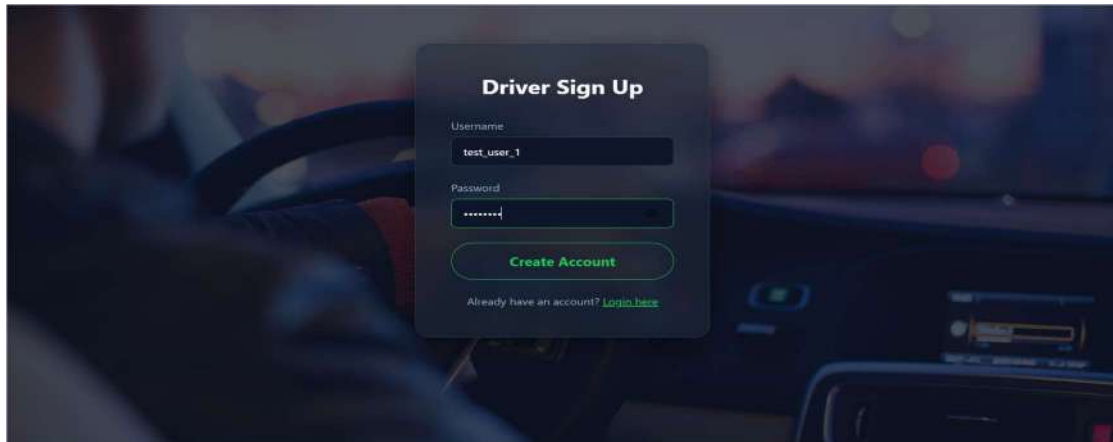
10.2 Functional Limitations

- There is no automatic reservation expiry. A slot reserved by a driver stays occupied until manually released or admin-overridden.
- There is no payment integration. Due fees are tracked in the database but settlement must be handled offline or via an admin fee reset.
- The system does not support concurrent multi-slot reservations (one active session per driver at a time).

10.3 Future Enhancements

- JWT-based authentication and server-side role middleware.
- Password hashing with bcrypt.
- Reservation expiry and automatic slot release via scheduled jobs.
- Analytics dashboard for lot usage and revenue trends.
- Payment gateway integration for online fee settlement.
- Mobile-optimised responsive UI.

10.4 Screenshots of Tests



My Vehicles

No vehicles registered.

Add New Vehicle

License Plate

KL 23 J 2134

Vehicle Type

Car

Make/Model

Creta

Add Vehicle

My Vehicles

KL 23 J 2134 Creta	CAR	Delete
KL 34 E 7896 Toyota Corrolla	CAR	Delete

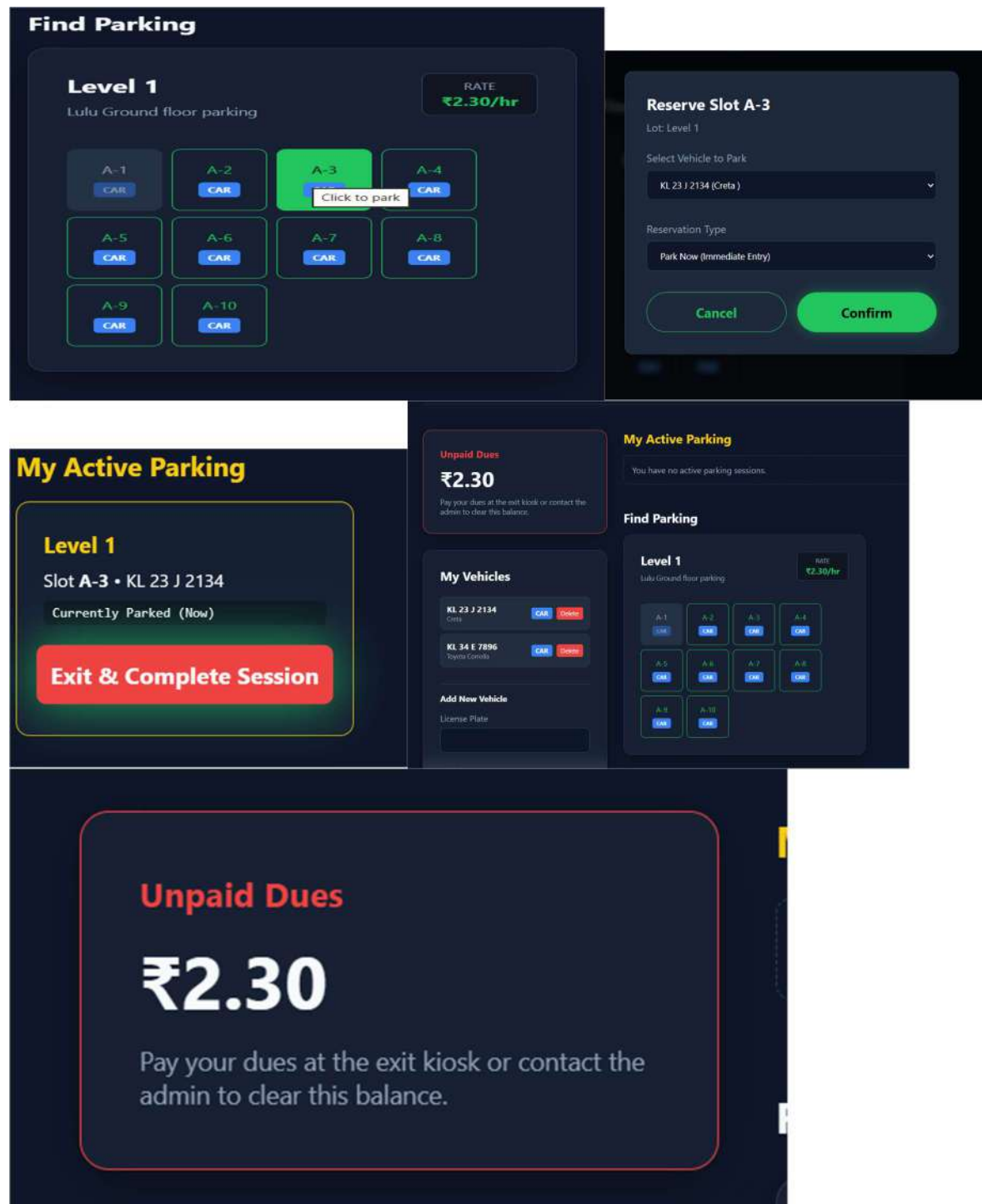
Find Parking

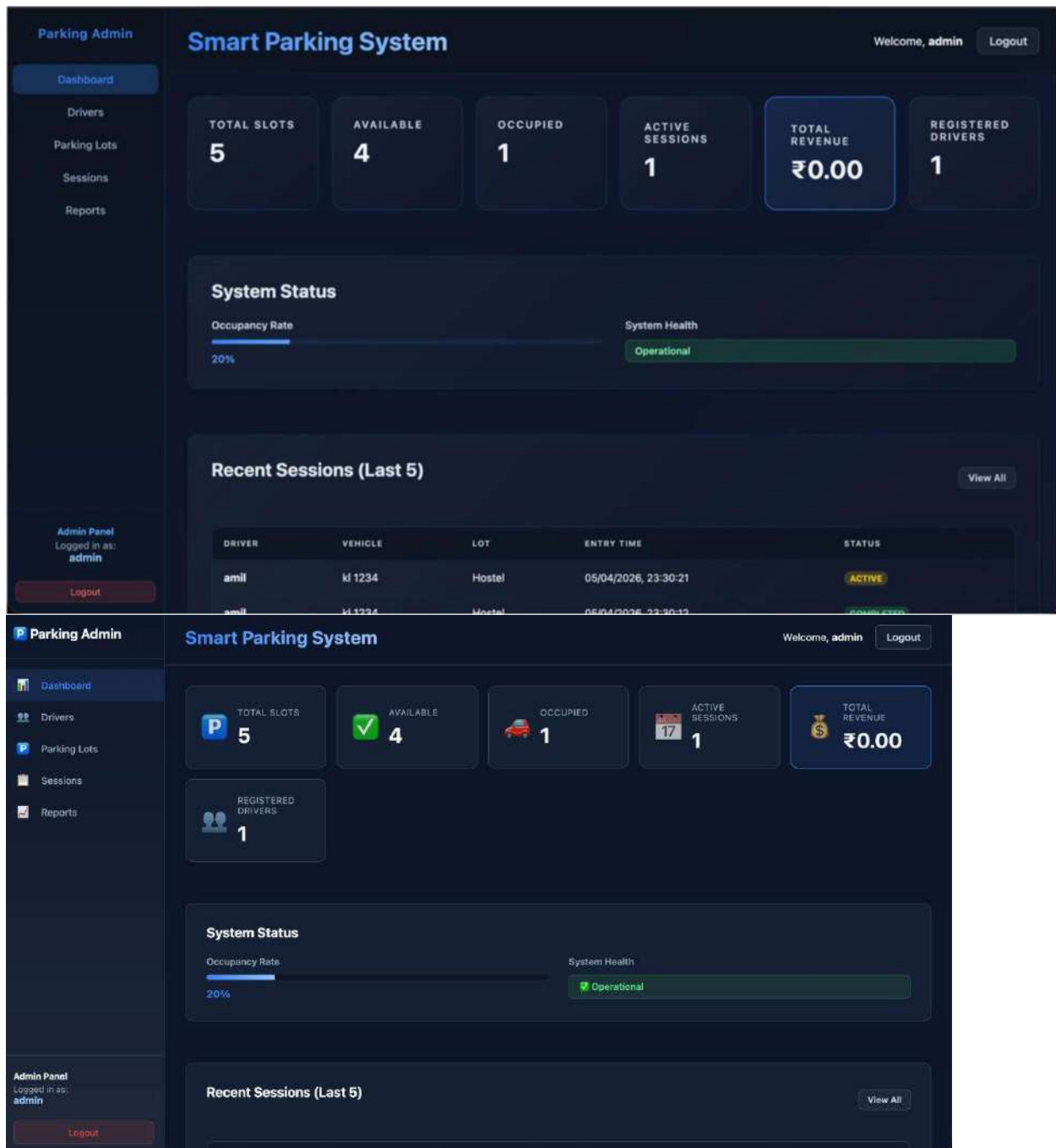
Level 1

Lulu Ground floor parking

RATE
₹2.30/hr

A-1 CAR	A-2 CAR	A-3 CAR	A-4 CAR
A-5 CAR	A-6 CAR	A-7 CAR	A-8 CAR
A-9 CAR	A-10 CAR		





Parking Admin

Dashboard

Drivers

Parking Lots

Sessions

Reports

Admin Panel
Logged in as: admin
Logout

Smart Parking System

Welcome, adminLogout

Add Parking Lot

Lot Name

Address

Generate Slots (Default: Car)

Price per Hour (₹)
0.00

Create Lot

Live Lot & Slot Configuration

Hostel

OCCUPANCY

Parking Admin

Dashboard

Drivers

Parking Lots

Sessions

Reports

Admin Panel
Logged in as: admin
Logout

Smart Parking System

Welcome, adminLogout

System Reports & Analytics

Bookings Summary

Lot Efficiency

Driver Insights

Session Details (All Records)

DATE	DRIVER	VEHICLE	DURATION	FEE	STATUS
05/04/2026	amil	kL 1234	-	-	ACTIVE
05/04/2026	amil	kL 1234	-	-	DONE

Parking Admin

Dashboard

Drivers

Parking Lots

Sessions

Reports

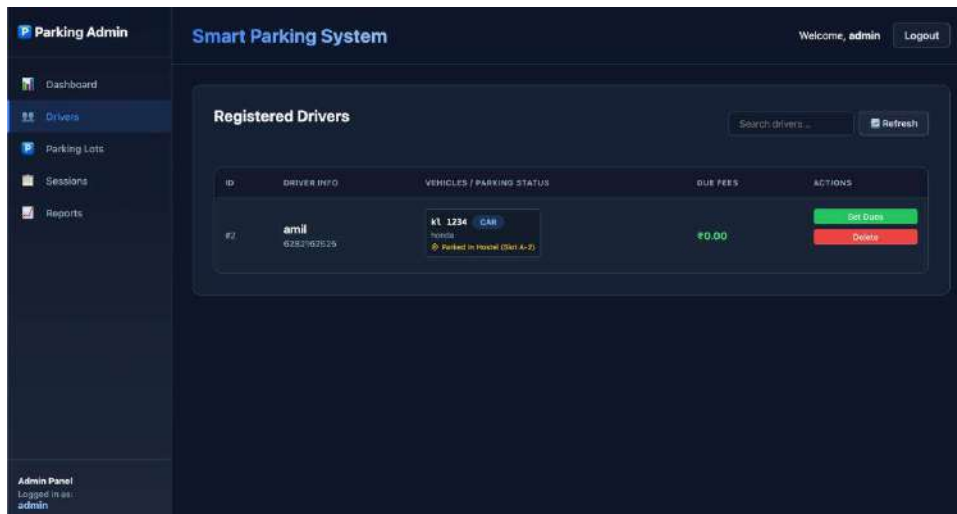
Admin Panel
Logged in as: admin
Logout

Smart Parking System

Welcome, adminLogout

Global Parking Sessions

#	USER	VEHICLE	LOT	SLOT NO	HOURS	FEE	ENTRY TIME	EXIT TIME	STATUS
#2	amil	kL 1234 (car)	Hostel	A-2	-	-	05/04/2026, 23:30:21	-	ACTIVE
#1	amil	kL 1234 (car)	Hostel	A-2	-	-	05/04/2026, 23:30:12	05/04/2026, 23:30:19	COMPLETED



References

- Express.js Documentation — <https://expressjs.com>
- MySQL2 npm Package — <https://www.npmjs.com/package/mysql2>
- Render Deployment Documentation — <https://render.com/docs>
- Clever Cloud MySQL Documentation — <https://www.clever-cloud.com/doc/deploy/addon/mysql>
- IEEE Software Test Documentation Standard (IEEE 829)
- Project Repository — <https://github.com/eliasjoby/Urban-Smart-Parking-System>